

Prim

April 20, 2026

0.1 Algoritmo di Prim - Implementazione e Testing

L'**Algoritmo di Prim** e' un algoritmo **Greedy** che costruisce l'*Albero di ricoprimento minimo incrementalmente*: ad ogni passo un **arco** e un **nodo** vengono aggiunti **irrevocabilmente**.

Viene visitato il *grafo*, prendendo ogni volta, fra tutti i nodi adiacenti a quelli gia' inclusi nella soluzione (quindi neri), quello connesso ad un nodo nero dall'arco di costo minimo.

Quest'algoritmo sfrutta una **Priority Queue** per memorizzare la **Frangia** di nodi adiacenti all'albero gia' costruito. Rispetto a una visita standard **BFS** (*Breadth-first search*), nella quale i nodi vengono visitati in base al loro ordine di inserimento nella *queue*, e' necessario *Prioritizzare* i nodi che **minimizzano** l'espansione dell'albero.

Iniziamo definendo la **struttura dati** adottata per l'implementazione dell'algoritmo, i quali contenuti verranno estratti da file di testo ASCII.

Liste di Adiacenza: Per ogni nodo memorizzo la lista dei nodi adiacenti. Si ottengono, per n nodi, m archi di un grafo non orientato, n liste di lunghezza $2m$. Viene scelta questa struttura in quanto e' piu' efficiente nel trovare i nodi adiacenti di un determinato nodo. Infatti avremo una complessita' di $O(\delta(u))$ per questa operazione fondamentale dell'algoritmo di **Prim**.

```
[19]: # Imports
import IPython
import warnings

from heapdict import heapdict

PATH_IN = "../data/Grafo-30-150.txt"
PATH_OUT = "../data/Grafo-30-150-OUT.txt"
```

```
[20]: def read_graph (fileName):
    with open(fileName, 'r') as f:
        curl = f.readlines() # read lines from file
        N = int(curl[0]) # Number of vertices

        # Lists (one for each node)
        adj = []
        for i in range(N):
            adj.append([])

        # Read File contents
```

```

    for i in range(1, len(curl)):
        l = curl[i]
        if len(l) > 0: # if the line is not empty
            u, v, c = l.split()
            u, v = int(u), int(v) # cast
            c = float(c)

            # Update Edges
            adj[u].append((v,c))
            adj[v].append((u,c))

    return N, adj
N, G = read_graph(PATH_IN) # Read and save Graph from File

```

Ora invece, seguendo lo *pseudocodice* presentato nelle note, implementiamo l'**Algoritmo di Prim**.

In questo caso ho deciso di utilizzare la libreria `heapdict` perche' fornisce un'implementazione di *heap binario* ordinato per chiave. Nell'algoritmo questo risulta estremamente comodo per via dell'ordinamento in base a `dist[u]`. Ho deciso di mantenere esplicitamente un array di *booleani* `visited[]`, piuttosto che usare il costrutto `in` fornito dalla libreria, per esser fedele il piu' possibile allo *pseudocodice*.

```

[21]: def Prim (G,s):
    # Init
    visited = [False] * N # marker array
    dist = [float('inf')] * N # distances array
    parent = [-1] * N # parent Array

    dist[s] = 0
    Q = heapdict() # Priority Queue
    for u in range(N):
        Q[u] = dist[u]

    while (len(Q) > 0):
        u, _ = Q.popitem() # getMin()
        visited[u] = True
        for v, c in G[u]:
            if (not visited[v] and c < dist[v]): # For each edge in adj
                parent[v] = u
                dist[v] = c
                Q[v] = c

    return dist, parent

```

Come richiesto nella traccia, l'output prodotto dovra' essere salvato su un file ASCII formattato.

```
[22]: def outFile (fileName, N, dist, parent):
    with open (fileName, 'w') as f:
        f.write(f"{N}\n") # Number of nodes
        for v in range(N):
            if parent[v] != -1: # Ignore s and all unreachable nodes
                u = parent[v]
                c = dist[v]
                f.write(f"{u} {v} {c}\n")
```

```
[23]: # main
s = 0
d, p = Prim(G, s)

outFile(PATH_OUT, N, d, p)
print("END\n")
```

END

Nella verifica di correttezza della mia implementazione ho trovato utile il confronto con un [web tool](#) che permette la simulazione del **minimum spanning tree**, allego di seguito le rappresentazioni grafiche dell'output.

Come analizzato nelle note, l'**Algoritmo di Prim** ha complessita' complessiva $O((n + m)\log n)$, infatti considerato un Grafo con n , m rispettivamente il numero di nodi e il numero di archi, le operazioni *dominanti* sono quelle sulla **Coda di Priorita'** che in questo caso, per `heapdict`, sono $O(\log n)$ sia per `push()` che `pop()`. Il ciclo `for` iniziale avra' pertanto costo $O(n\log n)$ mentre il ciclo `for` interno al `while`, eseguito al piu' n volte, viene eseguito complessivamente m volte e dunque il costo di aggiornamento della *coda di priorita'* ha costo $O(m\log n)$. Complessivamente dunque, $O((n + m)\log n)$. Potremmo inoltre dire che, come per l'**Algoritmo di Kruskal**, tale complessita' e' *asintoticamente* semplificabile in $O(m\log n)$ in quanto trattandosi di un *grafo connesso*, $m \geq n - 1$, $O(n + m) \sim O(m)$.

Baldini Filippo - 6393212